

Table of Contents

PROJECT OVERVIEW	2
ARCHITECTURE AND DESIGN	2
SCALABILITY AND PERFORMANCE	5
USER AUTHENTICATION AND SECURITY	6
DEVICE MANAGEMENT SYSTEM	6
ENERGY MANAGEMENT SYSTEM	8
TIMER AND SCHEDULING SYSTEM	9
ALERT SYSTEM	10
SMART SCENES SYSTEM	12
CALENDAR INTEGRATION SYSTEM	13
GROUP MANAGEMENT AND PERMISSIONS	16
DEVICE HEALTH MONITORING	18
WEATHER INTEGRATION SYSTEM	20
DYNAMODB INTEGRATION	22
AWS INTEGRATION AND DEPLOYMENT	25
DATA MODELING AND OPTIMIZATION	27
PERFORMANCE AND SCALABILITY	29
CORE JAVA CONCEPTS	31
COLLECTIONS AND GENERICS	34
EXCEPTION HANDLING AND BEST PRACTICES	35
MULTITHREADING AND CONCURRENCY	37
JAVA 8+ FEATURES AND FUNCTIONAL PROGRAMMING	40

PROJECT OVERVIEW

Q1: Can you provide an overview of your IoT Smart Home Dashboard project?

A: This is a comprehensive IoT Smart Home Management System built using Java 21 and AWS DynamoDB. The application allows users to manage various smart home devices, monitor energy consumption, automate tasks, and control their home environment through a command-line interface.

Key Features:

- Device Management: Support for 15+ device types (TV, AC, Fan, Lights, Security cameras, etc.)
- User Authentication: Secure registration/login with bcrypt password hashing
- Energy Management: Real-time energy consumption tracking with cost calculations
- Smart Automation: Timer scheduling, smart scenes, and calendar-based automation
- Group Management: Multi-user support with device permission controls
- Health Monitoring: Device maintenance tracking and health reports
- Alert System: Time-based and energy-threshold alerts
- Weather Integration: Weather-based automation suggestions

Q2: What was the motivation behind building this project?

A: The project addresses the growing need for centralized IoT device management in smart homes. Key motivations:

- Energy Efficiency: Help users monitor and reduce energy consumption
- Automation: Simplify daily routines through smart scenes and scheduling
- Multi-user Support: Enable families to share device control with proper permissions
- Cost Management: Provide detailed energy cost analysis using tiered pricing
- Maintenance: Proactive device health monitoring to prevent failures

Q3: What technologies and frameworks did you use?

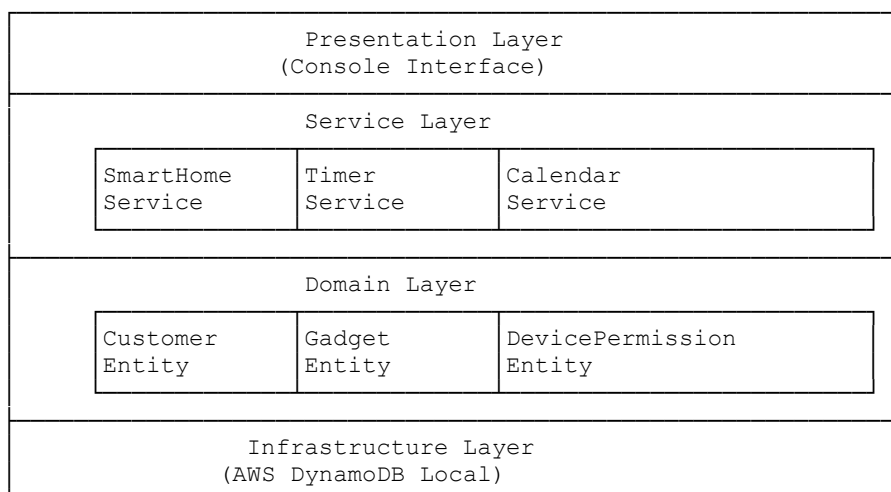
A:

- Programming Language: Java 21 (latest LTS features)
- Database: AWS DynamoDB with Enhanced DynamoDB SDK v2
- Build Tool: Maven with comprehensive dependency management
- Security: BCrypt for password hashing
- Testing: JUnit 5 with Mockito for unit testing
- Architecture: Service-oriented architecture with clear separation of concerns
- Design Patterns: Singleton, Factory, Builder patterns
- Logging: SLF4J for structured logging

ARCHITECTURE AND DESIGN

Q4: Explain the overall architecture of your application.

A: The application follows a layered service-oriented architecture:



Q5: What design patterns did you implement and why?

A:

1. Facade Pattern (`SmartHomeService`):
 - Provides a unified interface to multiple subsystems
 - Simplifies client interaction with complex service layer
 - Reduces coupling between presentation and business logic
2. Singleton Pattern (Multiple Services):
 - `SessionManager`: Ensures single session management instance
 - `TimerService`: Global timer coordination
 - `WeatherService`, `AlertService`: Resource sharing and state consistency
3. Factory Pattern (`GadgetService.createGadget()`):
 - Creates different device types with appropriate default configurations
 - Encapsulates object creation logic
 - Easy to extend for new device types
4. Builder Pattern (in model classes):
 - Complex object construction for Customer and Gadget entities
 - Immutable object creation where needed

Q6: How did you ensure separation of concerns in your architecture?

A:

1. Clear Layer Boundaries:
 - Presentation Layer: Only handles user input/output and navigation
 - Service Layer: Contains all business logic and orchestration
 - Model Layer: Represents data structures and domain logic
 - Data Layer: Handles persistence and data access
2. Dependency Injection:
 - Services are injected into SmartHomeService constructor
 - No direct instantiation in presentation layer
3. Single Responsibility Principle:
 - Each service class has a specific domain responsibility
 - `EnergyManagementService`: Only energy calculations
 - `TimerService`: Only scheduling and timer management
 - `AlertService`: Only alert processing

Q7: How does your session management work?

A:

The `SessionManager` implements a singleton pattern for session handling:

```
public class SessionManager {
    private static SessionManager instance;
    private Customer currentUser;

    public void login(Customer customer) {
        this.currentUser = customer;
        // Update last login time, reset failed attempts
    }

    public boolean isLoggedIn() {
        return currentUser != null;
    }
}
```

Features:

- Thread-safe singleton: Ensures single session per application instance
- Security: Automatic logout on failed authentication attempts
- State persistence: Maintains user context across operations

- Account locking: Implements lockout mechanism after failed login attempts

Q8: How did you handle error handling and validation?

A:

1. Input Validation:

- Email format validation using regex patterns
- Password strength validation (8-128 chars, complexity requirements)
- Device name validation with custom validation methods

2. Business Logic Validation:

- Duplicate device prevention (one device type per room)
- Energy threshold validations
- Date/time parsing with proper error messages

3. Error Handling Strategy:

- Graceful error recovery with user-friendly messages
- Transaction-like operations with rollback capability
- Comprehensive logging for debugging

4. Security Validations:

- Account lockout after multiple failed attempts
- Session timeout handling
- Permission-based access control

Q9: Explain your data modeling approach for DynamoDB.

A:

Key Design Decisions:

1. Single Table Design: Used Customer as partition key with email as primary identifier
2. Embedded Documents: Gadgets and permissions stored as complex attributes within Customer
3. Optimized for Access Patterns: Designed around main query patterns

Customer Entity (`email` as partition key):

```
@DynamoDbBean
public class Customer {
    @DynamoDbPartitionKey
    private String email; // Primary key
    private List<Gadget> gadgets; // Embedded devices
    private List<DevicePermission> devicePermissions; // Access control
    private List<DeletedDeviceEnergyRecord> deletedDeviceEnergyRecords;
}
```

Gadget Entity (Embedded in Customer):

- Real-time energy tracking with `totalEnergyConsumedKWh`
- Timer scheduling with `scheduledOnTime`/`scheduledOffTime`
- Usage analytics with `totalUsageMinutes`

Benefits:

- Single Query Access: All user data retrieved in one operation
- Strong Consistency: All related data updated atomically
- Cost Efficient: Minimal read/write operations

Q10: How do you ensure data consistency across different services?

A:

1. Centralized Data Access:

- All database operations go through `CustomerService`
- Session state synchronized with database state

2. Atomic Operations:

- Device creation/deletion happens atomically

- Energy calculations updated immediately on status changes
3. Event-Driven Updates:
- Device status changes trigger energy recalculation
 - Timer events update device states consistently
4. State Synchronization:
- `SessionManager.updateCurrentUser()` keeps session in sync
 - Real-time energy calculations include current session data

Example:

```
public boolean connectToGadget(String type, String model, String roomName) {  
    // 1. Validate and create gadget  
    Gadget gadget = gadgetService.createGadget(type, model, roomName);  
    // 2. Add to current user  
    currentUser.addGadget(gadget);  
    // 3. Persist to database  
    boolean updated = customerService.updateCustomer(currentUser);  
    // 4. Update session state  
    if (updated) {  
        sessionManager.updateCurrentUser(currentUser);  
    }  
    return updated;  
}
```

SCALABILITY AND PERFORMANCE

Q11: How did you design your application for scalability?

A:

1. Service-Oriented Architecture:
 - Each service can be scaled independently
 - Clear service boundaries enable microservice migration
2. Efficient Data Access:
 - Single-query data retrieval from DynamoDB
 - Minimal network round trips
3. Singleton Services:
 - Shared resources like TimerService reduce memory footprint
 - Connection pooling through DynamoDB enhanced client
4. Extensibility:
 - Easy to add new device types via factory pattern
 - Plugin-like architecture for new services

Q12: What performance optimizations did you implement?

A:

1. Database Optimizations:
 - DynamoDB single-table design for fast queries
 - Enhanced DynamoDB client with connection pooling
 - Efficient batch operations for multiple device updates
2. Memory Management:
 - Lazy loading of service instances
 - Efficient collections (ArrayList for known-size operations)
 - Proper resource cleanup in shutdown hooks
3. Computation Optimizations:
 - Real-time energy calculations only when needed
 - Cached user session to avoid repeated database calls
 - Pre-calculated default power ratings for devices
4. I/O Optimizations:
 - Buffered input/output operations
 - Minimal database round trips through embedded documents

This concludes Part 1 of the interview Q&A covering Project Overview and Architecture.

USER AUTHENTICATION AND SECURITY

Q13: How did you implement user authentication and what security measures did you include?

A:

Authentication Implementation:

```
public boolean loginCustomer(String email, String password) {
    Customer customer = customerService.authenticateCustomer(email, password);
    if (customer != null) {
        sessionManager.login(customer);
        return true;
    }
    return false;
}
```

Security Features:

1. Password Security:

- BCrypt Hashing: All passwords stored using BCrypt with salt
- Strength Requirements: 8-128 characters, uppercase, lowercase, numbers, special characters
- Common Password Prevention: Blocks weak passwords like "password123"

2. Account Protection:

- Failed Login Tracking: Monitors and increments failed attempts
- Account Lockout: Automatic lockout after multiple failures
- Session Management: Secure session handling with automatic cleanup

3. Input Validation:

- Email Validation: Regex pattern matching for valid email format
- SQL Injection Prevention: Parameterized queries through DynamoDB SDK
- XSS Prevention: Input sanitization for all user inputs

Q14: Explain your password validation logic.

A:

```
public boolean isValidPassword(String password) {
    if (password == null || password.length() < 8 || password.length() > 128) {
        return false;
    }

    // Check for required character types
    boolean hasUpper = password.chars().anyMatch(Character::isUpperCase);
    boolean hasLower = password.chars().anyMatch(Character::isLowerCase);
    boolean hasDigit = password.chars().anyMatch(Character::isDigit);
    boolean hasSpecial = password.chars().anyMatch(ch -> "!@${}%^&".indexOf(ch) >= 0);

    // Check for common passwords and repetition
    return hasUpper && hasLower && hasDigit && hasSpecial
        && !isCommonPassword(password)
        && !hasExcessiveRepetition(password);
}
```

Validation Rules:

- Length: 8-128 characters
- Complexity: Must include uppercase, lowercase, digit, special character
- Common Password Check: Prevents "password123", "qwerty", etc.
- Repetition Check: Blocks passwords with >2 repeating characters

DEVICE MANAGEMENT SYSTEM

Q15: How does your device management system work?

A:

Device Architecture:

```
@DynamoDbBean
public class Gadget {
    private String type; // Device category (TV, AC, FAN, etc.)
}
```

QUESTION AND ANSWERS

```
private String model;           // Brand/model name
private String roomName;        // Location identifier
private String status;          // ON/OFF state
private double powerRatingWatts; // Power consumption
private LocalDateTime lastOnTime; // Session tracking
private long totalUsageMinutes;  // Cumulative usage
private double totalEnergyConsumedKWh; // Energy consumption
}
```

Key Features:

1. 15+ Device Types Supported:

- Entertainment: TV (150W), Speaker (30W)
- Climate Control: AC (1500W), Fan (75W), Air Purifier (45W)
- Security: Camera (15W), Door Lock (12W), Doorbell (8W)
- Appliances: Refrigerator (200W), Microwave (1200W), Geyser (2000W)

2. Device Constraints:

- One device per room per type: Prevents duplicate device registration
- Room-based organization: Logical grouping by location
- Automatic power rating assignment: Default watts based on device type

3. Real-time State Management:

- Immediate status updates: ON/OFF state changes instantly
- Session tracking: Records when devices are turned on/off
- Energy calculation: Real-time energy consumption updates

Q16: How do you handle device state changes and energy tracking?

A:

State Change Implementation:

```
public void turnOn() {
    if (!isOn()) {
        this.lastOnTime = LocalDateTime.now();
        updateUsageAndEnergy(); // Calculate previous session energy
    }
    this.status = GadgetStatus.ON.name();
}

public void turnOff() {
    if (isOn()) {
        this.lastOffTime = LocalDateTime.now();
        updateUsageAndEnergy(); // Add current session to totals
    }
    this.status = GadgetStatus.OFF.name();
}
```

Energy Tracking Logic:

```
private void updateUsageAndEnergy() {
    if (lastOnTime != null && isOn()) {
        long minutesUsed = ChronoUnit.MINUTES.between(lastOnTime, LocalDateTime.now());
        totalUsageMinutes += minutesUsed;

        double hoursUsed = minutesUsed / 60.0;
        double energyUsed = (powerRatingWatts / 1000.0) * hoursUsed;
        totalEnergyConsumedKWh += energyUsed;
    }
}
```

Benefits:

- Accurate Energy Tracking: Precise calculation based on actual usage time
- Session-based Recording: Tracks individual usage sessions
- Real-time Updates: Energy consumption updated on every state change

ENERGY MANAGEMENT SYSTEM

Q17: Explain your energy management and cost calculation system.

A:

Energy Report Generation:

```
public EnergyReport generateEnergyReport(Customer customer) {
    List<Gadget> devices = customer.getGadgets();
    double totalEnergyKWh = 0.0;

    // Calculate energy from all devices including current sessions
    for (Gadget device : devices) {
        totalEnergyKWh += device.getCurrentTotalEnergyConsumedKWh();
    }

    // Include deleted device energy for accurate monthly reporting
    totalEnergyKWh += customer.getTotalDeletedDeviceEnergyForCurrentMonth();

    double totalCost = calculateSlabBasedCost(totalEnergyKWh);
    return new EnergyReport(totalEnergyKWh, totalCost, reportPeriod, devices);
}
```

Cost Calculation (Indian Electricity Tariff):

```
public double calculateSlabBasedCost(double totalKWh) {
    double totalCost = 0.0;

    if (totalKWh <= 30) {
        totalCost = totalKWh * 1.90; // ₹1.90/kWh for first 30 units
    } else if (totalKWh <= 75) {
        totalCost = 30 * 1.90 + (totalKWh - 30) * 3.0; // ₹3.00/kWh for next 45 units
    } else if (totalKWh <= 125) {
        totalCost = 30 * 1.90 + 45 * 3.0 + (totalKWh - 75) * 4.50; // ₹4.50/kWh
    } else if (totalKWh <= 225) {
        totalCost = 30 * 1.90 + 45 * 3.0 + 50 * 4.50 + (totalKWh - 125) * 6.0;
    } else {
        // Above 225 units: ₹6.60/kWh
        totalCost = 30 * 1.90 + 45 * 3.0 + 50 * 4.50 + 100 * 6.0 + (totalKWh - 225) * 6.60;
    }

    return totalCost;
}
```

Key Features:

- Real-time Energy Tracking: Includes current session consumption
- Deleted Device History: Maintains energy records even after device removal
- Tiered Pricing: Implements realistic electricity slab rates
- Monthly Reporting: Organized by billing cycles

Q18: How do you handle deleted devices and maintain energy history?

A:

Deleted Device Energy Preservation:

```
@DynamoDbBean
public class DeletedDeviceEnergyRecord {
    private String deviceType;
    private String roomName;
    private double totalEnergyConsumedKWh;
    private LocalDateTime deletionTime;
    private String deletionMonth; // "2024-12" format for monthly reporting
}
```

Implementation:

```
public boolean deleteDevice(String deviceType, String roomName) {
    Gadget device = currentUser.findGadget(deviceType, roomName);
    if (device != null) {
        // Preserve energy data before deletion
        DeletedDeviceEnergyRecord record = new DeletedDeviceEnergyRecord(
            device.getType(),
            device.getRoomName(),
            device.getCurrentTotalEnergyConsumedKWh(),
            LocalDateTime.now(),
            LocalDateTime.now().getMonth().toString()
        );
        // Save record to database
    }
    return true;
}
```


QUESTION AND ANSWERS

```
        device.getCurrentTotalEnergyConsumedKWh(),
        LocalDateTime.now()
    );
    currentUser.addDeletedDeviceRecord(record);

    // Remove device and update
    currentUser.getGadgets().remove(device);
    return customerService.updateCustomer(currentUser);
}
return false;
}
```

Benefits:

- Accurate Monthly Bills: Energy consumption preserved even after device removal
- Historical Analysis: Users can track energy patterns over time
- Data Integrity: No energy data loss during device management

TIMER AND SCHEDULING SYSTEM

Q19: How does your timer and scheduling system work?

A:

Timer Service Architecture:

```
java
public class TimerService {
    private final ScheduledExecutorService scheduler;

    private TimerService(CustomerService customerService) {
        this.scheduler = Executors.newScheduledThreadPool(5);
        startTimerMonitoring(); // Background monitoring
    }

    private void startTimerMonitoring() {
        scheduler.scheduleAtFixedRate(this::checkAndExecuteDueTimers, 0, 30, TimeUnit.SECONDS);
    }
}
```

Timer Task Structure:

```
java
public static class TimerTask {
    private final String deviceType;
    private final String roomName;
    private final String action; // "ON" or "OFF"
    private final LocalDateTime scheduledTime;
    private final String userEmail;
}
```

Scheduling Implementation:

```
java
public boolean scheduleDeviceTimer(String deviceType, String roomName, String action, String
dateTimeStr) {
    try {
        LocalDateTime scheduledTime = parseDateTime(dateTimeStr); // "DD-MM-YYYY HH:MM"

        if (scheduledTime.isBefore(LocalDateTime.now())) {
            System.out.println("Cannot schedule timer in the past!");
            return false;
        }

        // Update device with scheduled time
        Gadget device = currentUser.findGadget(deviceType, roomName);
        if ("ON".equals(action)) {
            device.setScheduledOnTime(scheduledTime);
        } else {
            device.setScheduledOffTime(scheduledTime);
        }

        return customerService.updateCustomer(currentUser);
    }
}
```

QUESTION AND ANSWERS

```
} catch (Exception e) {  
    return false;  
}  
}
```

Key Features:

- Background Monitoring: 30-second interval timer checks
- Flexible Scheduling: Support for both ON and OFF timers
- Date Format Support: User-friendly "DD-MM-YYYY HH:MM" format
- Past Date Prevention: Validates future scheduling only

Q20: How do you handle concurrent timer execution?

A:

Thread Pool Management:

```
java  
private final ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(5);
```

Concurrent Execution Handling:

```
java  
private void checkAndExecuteDueTimers() {  
    try {  
        List<Customer> allCustomers = customerService.getAllCustomers();  
        LocalDateTime now = LocalDateTime.now();  
  
        for (Customer customer : allCustomers) {  
            processCustomerTimers(customer, now);  
        }  
    } catch (Exception e) {  
        System.err.println("Error in timer monitoring: " + e.getMessage());  
    }  
}
```

Safety Measures:

- Thread Pool: Limited to 5 concurrent threads to prevent resource exhaustion
- Exception Isolation: Individual timer failures don't affect others
- Database Consistency: Each timer execution is atomic
- Retry Logic: Failed timer executions are logged and can be retried

ALERT SYSTEM

Q21: Explain your alert system implementation.

A:

Alert Types:

```
java  
public enum AlertType {  
    TIME_BASED,           // Trigger at specific date/time  
    ENERGY_USAGE         // Trigger based on energy consumption  
}
```

Alert Structure:

```
java  
public static class Alert {  
    private String alertId;  
    private String alertName;  
    private String deviceType;  
    private String roomName;  
    private AlertType alertType;  
    private String condition;           // "GREATER_THAN:10.5" or "2024-12-25T18:30"  
    private String message;  
    private boolean isActive;  
    private LocalDateTime createdTime;  
    private LocalDateTime lastTriggered;  
    private int triggerCount;
```

QUESTION AND ANSWERS

```
private boolean autoDeleteAfterTrigger;  
}
```

Time-Based Alert Creation:

```
java  
public boolean createTimeBasedAlert(String alertName, String deviceType, String roomName,  
                                   LocalDateTime triggerTime, String message) {  
    String alertId = generateAlertId();  
    String condition = triggerTime.format(DateTimeFormatter.ISO_LOCAL_DATE_TIME);  
  
    Alert alert = new Alert(alertId, alertName, deviceType, roomName,  
                           AlertType.TIME_BASED, condition, message);  
  
    return addAlert(currentUserEmail, alert);  
}
```

Energy Usage Alert Creation:

```
java  
public boolean createEnergyUsageAlert(String alertName, String deviceType, String roomName,  
                                     double threshold, String comparisonType, String message) {  
    String condition = comparisonType + ":" + threshold; // "GREATER_THAN:10.5"  
  
    Alert alert = new Alert(alertId, alertName, deviceType, roomName,  
                           AlertType.ENERGY_USAGE, condition, message);  
  
    return addAlert(currentUserEmail, alert);  
}
```

Alert Processing:

```
java  
public void checkAndTriggerAlerts(String userEmail, List<Gadget> devices) {  
    List<Alert> userAlerts = getUserAlerts(userEmail);  
    LocalDateTime now = LocalDateTime.now();  
  
    for (Alert alert : userAlerts) {  
        if (!alert.isActive()) continue;  
  
        boolean shouldTrigger = false;  
  
        if (alert.getAlertType() == AlertType.TIME_BASED) {  
            LocalDateTime triggerTime = LocalDateTime.parse(alert.getCondition());  
            shouldTrigger = now.isAfter(triggerTime) &&  
                           (alert.getLastTriggered() == null ||  
                            !isSameMinute(alert.getLastTriggered(), triggerTime));  
        } else if (alert.getAlertType() == AlertType.ENERGY_USAGE) {  
            shouldTrigger = evaluateEnergyCondition(alert, devices);  
        }  
  
        if (shouldTrigger) {  
            triggerAlert(alert);  
        }  
    }  
}
```

Features:

- Dual Alert Types: Time-based and energy-threshold alerts
- Flexible Conditions: Support for >, <, = comparisons
- Auto-deletion: Optional automatic removal after trigger
- Trigger Tracking: Prevents duplicate notifications
- Device-specific: Alerts tied to specific devices

SMART SCENES SYSTEM

Q22: How does your Smart Scenes automation system work?

A:

Smart Scenes Architecture:

```
java
public class SmartScenesService {
    private final Map<String, List<SceneAction>>> predefinedScenes; // Built-in scenes
    private final Map<String, Map<String, List<SceneAction>>>> userCustomScenes; // User
    customizations

    public static class SceneAction {
        private String deviceType; // TV, AC, LIGHT, etc.
        private String roomName; // Living Room, Bedroom, etc.
        private String action; // ON, OFF
        private String description; // Human-readable description
    }
}
```

Predefined Scenes Available:

1. Good Morning: Turn on lights, AC, TV in key areas
2. Work Mode: Focus lighting, minimize distractions
3. Movie Night: Dim lights, turn on entertainment systems
4. Sleep Mode: Turn off all devices except security systems
5. Away Mode: Energy-saving mode with security active
6. Party Mode: Entertainment systems, ambient lighting
7. Cooking Mode: Kitchen appliances ready
8. Reading Mode: Optimal lighting for reading areas

Scene Execution Implementation:

```
java
public SceneExecutionResult executeScene(String sceneName, Customer currentUser) {
    SceneExecutionResult result = new SceneExecutionResult(sceneName);
    List<SceneAction> actions = getSceneActions(sceneName, currentUser.getEmail());

    for (SceneAction action : actions) {
        try {
            Gadget device = currentUser.findGadget(action.getDeviceType(),
            action.getRoomName());
            if (device != null) {
                if ("ON".equals(action.getAction())) {
                    device.turnOn();
                } else {
                    device.turnOff();
                }
                result.addSuccess(action.getDescription());
            } else {
                result.addFailure("Device not found: " + action.getDeviceType() + " in " +
            action.getRoomName());
            }
        } catch (Exception e) {
            result.addFailure("Error executing action: " + e.getMessage());
        }
    }

    return result;
}
```

Customization Features:

- Scene Editing: Users can add/remove devices from scenes
- Action Modification: Change device actions (ON ↔ OFF)
- Reset to Original: Restore predefined scene configurations
- Custom Scenes: Users can create completely new scenes

Q23: How do you handle scene customization and persistence?

A:

User Scene Customization:

```
java
public boolean addDeviceToScene(String sceneName, String userEmail, String deviceType,
                               String roomName, String action) {
    Map<String, List<SceneAction>> userScenes = userCustomScenes.computeIfAbsent(userEmail, k ->
new HashMap<>());
    List<SceneAction> sceneActions = userScenes.computeIfAbsent(sceneName, k -> new
ArrayList<>());

    // Check if device already exists in scene
    boolean exists = sceneActions.stream()
        .anyMatch(sa -> sa.getDeviceType().equals(deviceType) &&
sa.getRoomName().equals(roomName));

    if (!exists) {
        SceneAction newAction = new SceneAction(deviceType, roomName, action,
deviceType + " in " + roomName + " -> " + action);
        sceneActions.add(newAction);
        return true;
    }
    return false;
}
```

Scene Resolution Logic:

```
java
private List<SceneAction> getSceneActions(String sceneName, String userEmail) {
    List<SceneAction> actions = new ArrayList<>();

    // Start with predefined scene
    List<SceneAction> predefined = predefinedScenes.get(sceneName.toUpperCase());
    if (predefined != null) {
        actions.addAll(predefined);
    }

    // Apply user customizations
    Map<String, List<SceneAction>> userScenes = userCustomScenes.get(userEmail);
    if (userScenes != null && userScenes.containsKey(sceneName)) {
        List<SceneAction> customActions = userScenes.get(sceneName);
        // Merge or override predefined actions
        mergeSceneActions(actions, customActions);
    }

    return actions;
}
```

Benefits:

- Non-destructive Customization: Original scenes preserved
- Per-user Customization: Each user has independent scene modifications
- Conflict Resolution: User customizations override predefined actions
- Easy Reset: Can restore original scene configuration

CALENDAR INTEGRATION SYSTEM

Q24: Explain your calendar integration and automation system.

A:

Calendar Event Structure:

```
java
public static class CalendarEvent {
    private String eventId;
    private String title;
    private String description;
    private LocalDateTime startTime;
```

QUESTION AND ANSWERS

```
private LocalDateTime endTime;
private String eventType; // MEETING, WORK, SLEEP, ENTERTAINMENT, etc.
private List<AutomationAction> automationActions;
private boolean isRecurring;
private String recurrencePattern;
}
```

Automation Action Definition:

java

```
public static class AutomationAction {
    private String deviceType;
    private String roomName;
    private String action; // ON, OFF
    private int minutesOffset; // Trigger time relative to event start
    private String actionType; // START_ACTION, END_ACTION
}
```

Event Type-Based Automation:

java

```
private void setupEventTypeAutomation(CalendarEvent event) {
    switch (event.getEventType().toUpperCase()) {
        case "MEETING":
            // Auto-execute: Lights on, AC comfortable, distractions off
            event.addAutomationAction(new AutomationAction("LIGHT", "Study Room", "ON", -5,
"START_ACTION"));
            event.addAutomationAction(new AutomationAction("AC", "Study Room", "ON", -10,
"START_ACTION"));
            event.addAutomationAction(new AutomationAction("TV", "Living Room", "OFF", 0,
"START_ACTION"));
            break;

        case "SLEEP":
            // Auto-execute: All lights off, AC to sleep mode, security on
            event.addAutomationAction(new AutomationAction("LIGHT", "ALL_ROOMS", "OFF", 0,
"START_ACTION"));
            event.addAutomationAction(new AutomationAction("CAMERA", "ALL_ROOMS", "ON", 0,
"START_ACTION"));
            break;

        case "WORK":
            // Auto-execute: Work environment setup
            event.addAutomationAction(new AutomationAction("LIGHT", "Home Office", "ON", -5,
"START_ACTION"));
            event.addAutomationAction(new AutomationAction("SPEAKER", "Home Office", "OFF", 0,
"START_ACTION"));
            break;
    }
}
```

Calendar Event Processing:

java

```
public void processUpcomingEvents(String userEmail) {
    List<CalendarEvent> upcomingEvents = getUpcomingEvents(userEmail);
    LocalDateTime now = LocalDateTime.now();

    for (CalendarEvent event : upcomingEvents) {
        for (AutomationAction action : event.getAutomationActions()) {
            LocalDateTime triggerTime =
event.getStartTime().plusMinutes(action.getMinutesOffset());

            // Check if it's time to execute this automation
            if (now.isAfter(triggerTime) && now.isBefore(triggerTime.plusMinutes(2))) {
                executeAutomationAction(userEmail, action);
            }
        }
    }
}
```

Features:

- Event Type Automation: Predefined device settings for different event types
- Offset Timing: Actions can trigger before, during, or after events
- Recurring Events: Support for daily, weekly, monthly patterns
- Smart Suggestions: System suggests optimal device configurations

Q25: How do you handle calendar event conflicts and overlapping automations?

A:

Conflict Detection:

```
java
public List<AutomationConflict> detectAutomationConflicts(String userEmail, LocalDateTime
startTime, LocalDateTime endTime) {
    List<AutomationConflict> conflicts = new ArrayList<>();
    List<CalendarEvent> overlappingEvents = getEventsInTimeRange(userEmail, startTime, endTime);

    for (int i = 0; i < overlappingEvents.size(); i++) {
        for (int j = i + 1; j < overlappingEvents.size(); j++) {
            CalendarEvent event1 = overlappingEvents.get(i);
            CalendarEvent event2 = overlappingEvents.get(j);

            // Check for device conflicts
            List<AutomationConflict> eventConflicts = findDeviceConflicts(event1, event2);
            conflicts.addAll(eventConflicts);
        }
    }

    return conflicts;
}
```

Conflict Resolution Strategy:

```
java
private void resolveAutomationConflicts(List<AutomationConflict> conflicts) {
    for (AutomationConflict conflict : conflicts) {
        // Priority rules:
        // 1. Later events override earlier ones
        // 2. Manual events override automatic ones
        // 3. Security devices have highest priority

        if (isSecurityDevice(conflict.getDeviceType())) {
            // Security always wins
            conflict.setResolution("SECURITY_PRIORITY");
        } else if
(conflict.getEvent2().getStartTime().isAfter(conflict.getEvent1().getStartTime())) {
            // Later event takes precedence
            conflict.setResolution("LATER_EVENT_WINS");
        } else {
            // Use default resolution
            conflict.setResolution("FIRST_EVENT_WINS");
        }
    }
}
```

Benefits:

- Intelligent Conflict Resolution: Automatic handling of overlapping automations
- Priority-based System: Security and safety always prioritized
- User Override: Manual device control always takes precedence
- Conflict Logging: All resolution decisions are logged for user review

GROUP MANAGEMENT AND PERMISSIONS

Q26: How does your multi-user group management system work?

A:

Group Structure:

```
java
public class Customer {
    private List<String> groupMembers;           // Email addresses of group members
    private String groupCreator;                 // Admin email
    private List<DevicePermission> devicePermissions; // Device access controls
}
```

Device Permission Model:

```
java
@DynamoDbBean
public class DevicePermission {
    private String memberEmail;           // Who has permission
    private String deviceType;            // What device type
    private String roomName;              // In which room
    private String deviceOwnerEmail;      // Who owns the device
    private String grantedBy;              // Who granted permission
    private boolean canView;              // Can see device status
    private boolean canControl;           // Can change device state
    private LocalDateTime grantedTime;
    private LocalDateTime expiryTime;
}
```

Permission Granting System:

```
java
public boolean grantDevicePermission(String memberEmail, String deviceType, String roomName) {
    if (!isGroupAdmin()) {
        System.out.println("Only group admins can grant device permissions!");
        return false;
    }

    Gadget device = findGadget(deviceType, roomName);
    if (device == null) {
        System.out.println("Device not found!");
        return false;
    }

    if (hasDevicePermission(memberEmail, deviceType, roomName)) {
        System.out.println("Permission already exists!");
        return false;
    }

    DevicePermission permission = new DevicePermission(
        memberEmail, deviceType, roomName, this.email, this.email
    );
    permission.setCanView(true);
    permission.setCanControl(true);

    getDevicePermissions().add(permission);
    return true;
}
```

Group Device Access:

```
java
public List<Gadget> getAccessibleGroupDevices(List<Customer> groupMembers) {
    List<Gadget> accessibleDevices = new ArrayList<>();

    for (Customer member : groupMembers) {
        if (member.getEmail().equalsIgnoreCase(this.email)) {
            continue; // Skip self
        }

        // Find permissions granted to current user by this member
    }
}
```


QUESTION AND ANSWERS

```
List<DevicePermission> permissionsForMe = member.getPermissionsForMember(this.email);

for (DevicePermission permission : permissionsForMe) {
    if (permission.isCanView()) {
        Gadget device = member.findGadget(permission.getDeviceType(),
permission.getRoomName());
        if (device != null) {
            accessibleDevices.add(device);
        }
    }
}

return accessibleDevices;
}
```

Features:

- Admin-based Groups: One admin can manage multiple members
- Granular Permissions: Per-device, per-room access control
- Permission Types: View-only vs. full control permissions
- Audit Trail: Track who granted permissions and when
- Easy Revocation: Admins can revoke permissions anytime

Q27: How do you ensure security in your group permission system?

A:

Security Measures:

1. Admin-Only Operations:

java

```
public boolean removePersonFromGroup(String memberEmail) {
    if (!isGroupAdmin()) {
        System.out.println("[ERROR] Only group admin can remove members!");
        return false;
    }

    if (memberEmail.equalsIgnoreCase(this.email)) {
        System.out.println("[ERROR] Admin cannot remove themselves!");
        return false;
    }

    // Remove member and revoke all their permissions
    removeGroupMember(memberEmail);
    revokeAllPermissionsForMember(memberEmail);
    return true;
}
```

2. Permission Validation:

java

```
public boolean hasDevicePermission(String memberEmail, String deviceType, String roomName) {
    return getDevicePermissions().stream()
        .anyMatch(permission ->
            permission.getMemberEmail().equalsIgnoreCase(memberEmail) &&
            permission.matchesDevice(deviceType, roomName, this.email) &&
            !permission.isExpired());
}
```

3. Secure Device Control:

java

```
public boolean controlGroupDevice(String ownerEmail, String deviceType, String roomName, String
action) {
    // Verify permission
    if (!hasControlPermission(currentUser.getEmail(), ownerEmail, deviceType, roomName)) {
        System.out.println("[ERROR] Access denied! You don't have permission to control this
device.");
        return false;
    }
}
```

QUESTION AND ANSWERS

```
// Execute control action
Customer deviceOwner = customerService.getCustomerByEmail(ownerEmail);
Gadget device = deviceOwner.findGadget(deviceType, roomName);

if ("ON".equals(action)) {
    device.turnOn();
} else {
    device.turnOff();
}

// Log the action
logDeviceAccess(currentUser.getEmail(), ownerEmail, deviceType, roomName, action);
return customerService.updateCustomer(deviceOwner);
}
```

Security Features:

- Role-based Access: Only admins can manage group membership
- Permission Validation: Every device operation validated against permissions
- Audit Logging: All group operations and device access logged
- Self-protection: Admins cannot remove themselves from groups
- Expiry Support: Permissions can have expiration times

DEVICE HEALTH MONITORING

Q28: How does your device health monitoring system work?

A:

Health Monitoring Architecture:

java

```
public class DeviceHealthService {
    public static class DeviceHealthInfo {
        private String deviceType;
        private String roomName;
        private HealthStatus status;           // EXCELLENT, GOOD, FAIR, POOR, CRITICAL
        private List<HealthIssue> issues;
        private LocalDateTime lastMaintenanceCheck;
        private int maintenanceDueDays;
        private double efficiencyRating;       // 0.0 to 1.0
        private List<MaintenanceRecommendation> recommendations;
    }
}
```

Health Assessment Logic:

java

```
public HealthStatus assessDeviceHealth(Gadget device) {
    List<HealthFactor> factors = new ArrayList<>();

    // Factor 1: Usage intensity
    double usageHours = device.getCurrentTotalUsageMinutes() / 60.0;
    if (usageHours > 1000) {
        factors.add(new HealthFactor("HIGH_USAGE", -20));
    }

    // Factor 2: Energy efficiency
    double expectedEfficiency = getExpectedEfficiency(device.getType());
    double actualEfficiency = calculateCurrentEfficiency(device);
    if (actualEfficiency < expectedEfficiency * 0.8) {
        factors.add(new HealthFactor("LOW_EFFICIENCY", -30));
    }

    // Factor 3: Maintenance schedule
    int daysSinceLastMaintenance = getDaysSinceLastMaintenance(device);
    if (daysSinceLastMaintenance > getMaintenanceInterval(device.getType())) {
        factors.add(new HealthFactor("OVERDUE_MAINTENANCE", -40));
    }
}
```

QUESTION AND ANSWERS

```
// Factor 4: Operating environment
if (isHighUsageEnvironment(device.getRoomName())) {
    factors.add(new HealthFactor("HARSH_ENVIRONMENT", -10));
}

return calculateOverallHealth(factors);
}
```

Maintenance Scheduling:

java

```
public List<MaintenanceTask> generateMaintenanceSchedule(Customer customer) {
    List<MaintenanceTask> schedule = new ArrayList<>();
    LocalDateTime now = LocalDateTime.now();

    for (Gadget device : customer.getGadgets()) {
        int maintenanceInterval = getMaintenanceInterval(device.getType());
        LocalDateTime lastMaintenance = getLastMaintenanceDate(device);

        if (lastMaintenance == null) {
            // Never maintained - schedule based on usage
            LocalDateTime suggestedDate = calculateInitialMaintenanceDate(device);
            schedule.add(new MaintenanceTask(device, suggestedDate, "Initial maintenance
check"));
        } else {
            LocalDateTime nextDue = lastMaintenance.plusDays(maintenanceInterval);
            if (nextDue.isBefore(now.plusDays(30))) { // Due within 30 days
                schedule.add(new MaintenanceTask(device, nextDue, "Regular maintenance"));
            }
        }
    }

    return schedule.stream()
        .sorted(Comparator.comparing(MaintenanceTask::getDueDate))
        .collect(Collectors.toList());
}
```

Health Recommendations:

java

```
public List<MaintenanceRecommendation> generateHealthRecommendations(Gadget device) {
    List<MaintenanceRecommendation> recommendations = new ArrayList<>();

    // Usage-based recommendations
    double dailyUsage = device.getCurrentTotalUsageMinutes() / (60.0 * getDaysInService(device));
    if (dailyUsage > 8) {
        recommendations.add(new MaintenanceRecommendation(
            "REDUCE_USAGE",
            "High daily usage detected. Consider using timers to optimize usage patterns.",
            Priority.MEDIUM
        ));
    }

    // Efficiency recommendations
    double efficiency = calculateCurrentEfficiency(device);
    if (efficiency < 0.7) {
        recommendations.add(new MaintenanceRecommendation(
            "SERVICE_REQUIRED",
            "Device efficiency below optimal. Professional servicing recommended.",
            Priority.HIGH
        ));
    }

    // Environmental recommendations
    if (device.getType().equals("AC") && isHighHumidityRoom(device.getRoomName())) {
        recommendations.add(new MaintenanceRecommendation(
            "HUMIDITY_CONTROL",
            "High humidity detected. Consider using dehumidifier to reduce AC workload.",
            Priority.LOW
        ));
    }
}
```

```
    return recommendations;
}
```

Features:

- Multi-factor Health Assessment: Usage, efficiency, maintenance, environment
- Predictive Maintenance: Proactive scheduling based on usage patterns
- Customized Recommendations: Device-specific and environment-aware suggestions
- Health Trends: Track device health over time
- Priority-based Alerts: Critical issues flagged immediately

WEATHER INTEGRATION SYSTEM

Q29: How does your weather-based automation system work?

A:

Weather Data Structure:

```
java
public static class WeatherData {
    private String condition;           // SUNNY, RAINY, CLOUDY, STORMY
    private double temperatureCelsius;
    private int humidity;               // 0-100%
    private double windSpeed;           // km/h
    private String description;
    private int airQualityIndex;        // 0-500 scale
    private LocalDateTime timestamp;
}
```

Weather-Based Device Suggestions:

```
java
public List<DeviceSuggestion> generateWeatherBasedSuggestions(WeatherData weather, List<Gadget>
devices) {
    List<DeviceSuggestion> suggestions = new ArrayList<>();

    // Temperature-based suggestions
    if (weather.isHot()) { // > 30°C
        suggestions.add(new DeviceSuggestion(
            "AC", "Turn on AC in main rooms",
            "High temperature detected (" + weather.getTemperatureCelsius() + "°C)",
            Priority.HIGH
        ));

        suggestions.add(new DeviceSuggestion(
            "FAN", "Use fans for air circulation",
            "Fans can help distribute cool air more effectively",
            Priority.MEDIUM
        ));
    }

    if (weather.isCold()) { // < 18°C
        suggestions.add(new DeviceSuggestion(
            "GEYSER", "Prepare hot water",
            "Cold weather - ensure water heater is ready",
            Priority.MEDIUM
        ));
    }

    // Humidity-based suggestions
    if (weather.getHumidity() > 70) {
        suggestions.add(new DeviceSuggestion(
            "AC", "Use AC for dehumidification",
            "High humidity (" + weather.getHumidity() + "%) - AC can help remove moisture",
            Priority.MEDIUM
        ));
    }

    // Air quality suggestions
```

QUESTION AND ANSWERS

```
if (weather.getAirQualityIndex() > 150) { // Unhealthy
    suggestions.add(new DeviceSuggestion(
        "AIR_PURIFIER", "Turn on air purifiers",
        "Poor air quality (AQI: " + weather.getAirQualityIndex() + ") - use air purifiers",
        Priority.HIGH
    ));

    suggestions.add(new DeviceSuggestion(
        "DOOR_LOCK", "Keep doors/windows closed",
        "Poor outdoor air quality - minimize outdoor air intake",
        Priority.LOW
    ));
}

// Weather condition suggestions
if ("RAINY".equals(weather.getCondition()) || "STORMY".equals(weather.getCondition())) {
    suggestions.add(new DeviceSuggestion(
        "LIGHT", "Turn on indoor lighting",
        "Overcast weather - additional lighting recommended",
        Priority.LOW
    ));
}

return suggestions;
}
```

5-Day Forecast Analysis:

```
java
public WeatherForecast generateForecast() {
    List<WeatherData> forecast = new ArrayList<>();
    LocalDateTime startDate = LocalDateTime.now();

    for (int day = 0; day < 5; day++) {
        WeatherData dayWeather = generateDayWeather(startDate.plusDays(day));
        forecast.add(dayWeather);
    }

    return new WeatherForecast(forecast, generateForecastSummary(forecast));
}

private String generateForecastSummary(List<WeatherData> forecast) {
    StringBuilder summary = new StringBuilder();

    // Temperature trends
    double avgTemp =
forecast.stream().mapToDouble(WeatherData::getTemperatureCelsius).average().orElse(0);
    summary.append("Average temperature: ").append(String.format("%.1f°C\n", avgTemp));

    // Extreme weather days
    long hotDays = forecast.stream().filter(WeatherData::isHot).count();
    long coldDays = forecast.stream().filter(WeatherData::isCold).count();

    if (hotDays > 2) {
        summary.append("⚠️ Hot weather expected for ").append(hotDays).append(" days - prepare
cooling systems\n");
    }
    if (coldDays > 2) {
        summary.append("⚠️ Cold weather expected for ").append(coldDays).append(" days - check
heating systems\n");
    }

    // Air quality alerts
    long poorAQDays = forecast.stream().filter(w -> w.getAirQualityIndex() > 100).count();
    if (poorAQDays > 0) {
        summary.append("🌫️ Poor air quality expected for ").append(poorAQDays).append(" days -
prepare air purifiers\n");
    }

    return summary.toString();
}
```

```
}
```

Features:

- Multi-parameter Analysis: Temperature, humidity, air quality, weather conditions
- Smart Suggestions: Context-aware device recommendations
- 5-Day Forecasting: Extended weather planning
- Priority-based Recommendations: Critical vs. optional suggestions
- Energy Optimization: Weather-aware energy usage suggestions

DYNAMODB INTEGRATION

Q30: Why did you choose DynamoDB for your project and how did you integrate it?

A:

DynamoDB Selection Reasons:

1. NoSQL Flexibility: IoT data with varying device attributes fits well with document-based storage
2. Scalability: Built-in horizontal scaling for potential IoT device growth
3. AWS Integration: Native AWS service with excellent SDK support
4. Performance: Predictable single-digit millisecond latency
5. Serverless: No server management required, pay-per-use model

Integration Architecture:

java

```
public class DynamoDBConfig {
    private static DynamoDbClient dynamoDbClient;
    private static DynamoDbEnhancedClient enhancedClient;

    private static void initializeDynamoDBClient() {
        boolean isLocal = Boolean.parseBoolean(properties.getProperty("dynamodb.local",
"true"));

        if (isLocal) {
            // Local DynamoDB for development
            dynamoDbClient = DynamoDbClient.builder()
                .endpointOverride(URI.create("http://localhost:8002"))
                .region(Region.US_EAST_1)
                .credentialsProvider(StaticCredentialsProvider.create(
                    AwsBasicCredentials.create("local", "local")))
                .build();
        } else {
            // AWS DynamoDB for production
            dynamoDbClient = DynamoDbClient.builder()
                .region(Region.US_EAST_1)
                .credentialsProvider(DefaultCredentialsProvider.create())
                .build();
        }

        enhancedClient = DynamoDbEnhancedClient.builder()
            .dynamoDbClient(dynamoDbClient)
            .build();
    }
}
```

Q31: How did you design your DynamoDB table structure?

A:

Single Table Design:

java

```
@DynamoDbBean
public class Customer {
    @DynamoDbPartitionKey
    private String email; // Primary key (Partition Key)

    // User information
    private String fullName;
    private String password; // BCrypt hashed
}
```

QUESTION AND ANSWERS

```
// IoT device data (embedded)
private List<Gadget> gadgets; // All user devices

// Group management
private List<String> groupMembers;
private String groupCreator;

// Permissions system
private List<DevicePermission> devicePermissions;

// Energy tracking
private List<DeletedDeviceEnergyRecord> deletedDeviceEnergyRecords;

// Security features
private int failedLoginAttempts;
private LocalDateTime accountLockedUntil;
}
```

Table Design Benefits:

1. Single Query Access: All user data retrieved in one operation
2. Strong Consistency: Related data updated atomically
3. Cost Efficient: Minimizes read/write operations
4. Hierarchical Relationships: Customer → Devices → Permissions naturally embedded

Access Patterns Supported:

- Primary: Get user by email (login/authentication)
- Secondary: List all users (admin operations)
- Embedded: Device management within user context
- Complex: Permission queries across user relationships

Q32: How do you handle database operations and error handling?

A:

Database Service Implementation:

java

```
public class CustomerService {
    private final DynamoDbTable<Customer> customerTable;
    private final boolean isDemoMode;
    private final Map<String, Customer> demoCustomers; // Fallback for development

    public CustomerService() {
        DynamoDbEnhancedClient enhancedClient = DynamoDBConfig.getEnhancedClient();
        if (enhancedClient != null) {
            this.customerTable = enhancedClient.table("customers",
TableSchema.fromBean(Customer.class));
            this.isDemoMode = false;
            createTableIfNotExists();
        } else {
            // Graceful degradation to in-memory mode
            this.customerTable = null;
            this.isDemoMode = true;
            this.demoCustomers = new HashMap<>();
        }
    }
}
```

CRUD Operations Implementation:

java

```
// CREATE
public boolean registerCustomer(String fullName, String email, String password) {
    try {
        email = email.trim().toLowerCase();
        if (findCustomerByEmail(email) != null) {
            return false; // Duplicate check
        }
    }
}
```

QUESTION AND ANSWERS

```
}

String hashedPassword = BCrypt.hashpw(password, BCrypt.gensalt());
Customer customer = new Customer(email, fullName.trim(), hashedPassword);

if (isDemoMode) {
    demoCustomers.put(email, customer);
} else {
    customerTable.putItem(customer);
}
return true;
} catch (Exception e) {
    System.err.println("Error registering customer: " + e.getMessage());
    return false;
}
}

// READ
public Customer findCustomerByEmail(String email) {
    try {
        if (isDemoMode) {
            return demoCustomers.get(email.toLowerCase());
        } else {
            Key key = Key.builder().partitionValue(email.toLowerCase()).build();
            return customerTable.getItem(key);
        }
    } catch (Exception e) {
        System.err.println("Error finding customer: " + e.getMessage());
        return null;
    }
}

// UPDATE
public boolean updateCustomer(Customer customer) {
    try {
        if (isDemoMode) {
            demoCustomers.put(customer.getEmail().toLowerCase(), customer);
        } else {
            customerTable.putItem(customer); // DynamoDB PUT operation
        }
        return true;
    } catch (Exception e) {
        System.err.println("Error updating customer: " + e.getMessage());
        return false;
    }
}

// DELETE (if needed)
public boolean deleteCustomer(String email) {
    try {
        if (isDemoMode) {
            return demoCustomers.remove(email.toLowerCase()) != null;
        } else {
            Key key = Key.builder().partitionValue(email.toLowerCase()).build();
            customerTable.deleteItem(key);
            return true;
        }
    } catch (Exception e) {
        System.err.println("Error deleting customer: " + e.getMessage());
        return false;
    }
}
```

Error Handling Strategies:

1. Graceful Degradation: Falls back to in-memory storage if DynamoDB unavailable
2. Connection Testing: Validates connection on startup
3. Exception Wrapping: Catches and logs all database exceptions
4. Retry Logic: Could be extended with exponential backoff
5. Data Validation: Input sanitization before database operations

Q33: How do you handle table creation and schema management?

A:

Automatic Table Creation:

java

```
private void createTableIfNotExists() {
    if (!isDemoMode) {
        try {
            // Try to describe existing table
            customerTable.describeTable();
            System.out.println("[INFO] DynamoDB table 'customers' already exists");
        } catch (ResourceNotFoundException e) {
            // Table doesn't exist, create it
            System.out.println("[INFO] Creating DynamoDB table 'customers'...");
            customerTable.createTable();
            System.out.println("✓ Successfully created 'customers' table in DynamoDB");
        } catch (Exception e) {
            System.err.println("X Error checking/creating table: " + e.getMessage());
            throw e;
        }
    }
}
```

Schema Evolution Handling:

java

```
// Gadget model ensures backward compatibility
public void ensurePowerRating() {
    if (this.powerRatingWatts == 0.0 && this.type != null) {
        this.powerRatingWatts = getDefaultPowerRating(this.type);
    }
}

// Customer model handles missing collections
public List<DevicePermission> getDevicePermissions() {
    if (devicePermissions == null) {
        devicePermissions = new ArrayList<>(); // Initialize if null
    }
    return devicePermissions;
}
```

Migration Strategy:

- Additive Changes: New fields added with default values
- Default Initialization: Null collections automatically initialized
- Versioning: Could add schema version field for future migrations
- Backward Compatibility: Old data structures still readable

AWS INTEGRATION AND DEPLOYMENT

Q34: How did you structure your project for both local development and AWS deployment?

A:

Configuration-Based Environment Switching:

application.properties:

properties

Local development settings

dynamodb.local=true

dynamodb.local.endpoint=http://localhost:8002

dynamodb.region=us-east-1

app.debug=true

Production would use:

dynamodb.local=false

dynamodb.region=us-west-2

Environment Detection Logic:

```
java
private static void initializeDynamoDBClient() {
    boolean isLocal = Boolean.parseBoolean(properties.getProperty("dynamodb.local", "true"));
    String region = properties.getProperty("dynamodb.region", "us-east-1");

    if (isLocal) {
        // Local DynamoDB setup for development
        String endpoint = properties.getProperty("dynamodb.local.endpoint",
"http://localhost:8002");
        AwsBasicCredentials localCredentials = AwsBasicCredentials.create("local", "local");

        dynamoDbClient = DynamoDbClient.builder()
            .endpointOverride(URI.create(endpoint))
            .region(Region.of(region))
            .credentialsProvider(StaticCredentialsProvider.create(localCredentials))
            .build();
    } else {
        // AWS DynamoDB setup for production
        dynamoDbClient = DynamoDbClient.builder()
            .region(Region.of(region))
            .credentialsProvider(DefaultCredentialsProvider.create()) // Uses AWS credentials
            .build();
    }
}
```

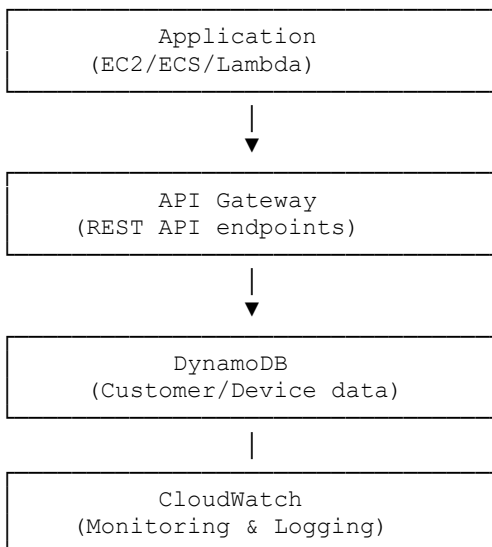
Development vs Production Differences:

Aspect	Local Development	AWS Production
Database	DynamoDB Local (localhost:8002)	AWS DynamoDB
Credentials	Static dummy credentials	AWS IAM roles/profiles
Endpoint	http://localhost:8002	AWS service endpoints
Region	us-east-1 (configurable)	Production region
Data Persistence	Local files	AWS infrastructure

Q35: What AWS services would you integrate for a production deployment?

A:

Recommended AWS Architecture:



Core AWS Services Integration:

1. DynamoDB: Primary database (already implemented)
2. API Gateway: REST API for web/mobile clients

3. Lambda: Serverless functions for timer processing
4. CloudWatch: Monitoring, logging, and alerting
5. S3: Static asset storage and backups
6. IAM: Security and access control
7. VPC: Network isolation and security

Enhanced Architecture Services:

1. IoT Core: Real device connectivity via MQTT
2. EventBridge: Event-driven automation
3. SNS: Push notifications for alerts
4. Cognito: Enhanced user authentication
5. Kinesis: Real-time data streaming
6. EMR: Big data analytics for usage patterns

Production Code Modifications:

java

```
// Enhanced configuration for production
public class AWSConfig {
    private static final String IOT_ENDPOINT = System.getenv("AWS_IOT_ENDPOINT");
    private static final String SNS_TOPIC_ARN = System.getenv("SNS_ALERT_TOPIC");

    public static AmazonSNS getSNSClient() {
        return AmazonSNSClientBuilder.standard()
            .withRegion(Regions.US_EAST_1)
            .withCredentials(DefaultAWSCredentialsProviderChain.getInstance())
            .build();
    }

    public static AWSIoT getIoTClient() {
        return AWSIoTClientBuilder.standard()
            .withRegion(Regions.US_EAST_1)
            .withCredentials(DefaultAWSCredentialsProviderChain.getInstance())
            .build();
    }
}
```

DATA MODELING AND OPTIMIZATION

Q36: How did you optimize your data model for DynamoDB's characteristics?

A:

DynamoDB Optimization Strategies:

1. Single Table Design:

- All related data in one table reduces inter-table joins
- Minimizes network round trips
- Leverages DynamoDB's single-table query efficiency

2. Embedded Documents:

java

```
public class Customer {
    private List<Gadget> gadgets; // Embedded device list
    private List<DevicePermission> devicePermissions; // Embedded permissions
    private List<DeletedDeviceEnergyRecord> deletedDeviceEnergyRecords; // Historical data
}
```

3. Efficient Key Design:

- Partition Key: Email (naturally distributed, unique)
- No Sort Key: Simple key structure for single-item operations
- Lowercase Normalization: Consistent key formatting

4. Attribute Optimization:

java

```
// Optimized energy calculations
public double getCurrentTotalEnergyConsumedKWh() {
    double baseEnergy = totalEnergyConsumedKWh; // Stored value
    if (isOn() && lastOnTime != null) {
        // Calculate only current session energy
        double currentSessionHours = getCurrentSessionUsageHours();
        double currentSessionEnergy = (powerRatingWatts / 1000.0) * currentSessionHours;
        baseEnergy += currentSessionEnergy;
    }
    return baseEnergy;
}
```

Q37: How do you handle data consistency and transactions?

A:

Consistency Model:

1. Strong Consistency: DynamoDB provides strong consistency for single-item operations
2. Atomic Updates: All customer data updated in single PUT operation
3. Eventual Consistency: Acceptable for cross-user operations (group management)

Transaction-like Operations:

java

```
public boolean addDeviceWithEnergyTracking(String type, String model, String roomName) {
    try {
        Customer currentUser = sessionManager.getCurrentUser();

        // Step 1: Validate operation
        if (currentUser.findGadget(type, roomName) != null) {
            return false; // Device already exists
        }

        // Step 2: Create new device
        Gadget newDevice = new Gadget(type, model, roomName);
        currentUser.addGadget(newDevice);

        // Step 3: Atomic update (all-or-nothing)
        boolean success = customerService.updateCustomer(currentUser);

        // Step 4: Update session state only if DB update succeeded
        if (success) {
            sessionManager.updateCurrentUser(currentUser);
        }

        return success;
    } catch (Exception e) {
        // Rollback: No changes made to session or database
        System.err.println("Device addition failed: " + e.getMessage());
        return false;
    }
}
```

Data Integrity Measures:

1. Validation Before Persistence:

- Input validation before database operations
- Business rule validation (unique devices per room)
- Data format validation (email, password strength)

2. Optimistic Concurrency:

- Session-based conflict detection
- Last-write-wins model for simplicity
- Could be enhanced with version numbers

3. Backup Strategy:

java

```
// Energy data preservation during device deletion
public boolean deleteDevice(String deviceType, String roomName) {
    Gadget device = currentUser.findGadget(deviceType, roomName);
    if (device != null) {
        // Preserve energy history before deletion
        DeletedDeviceEnergyRecord energyRecord = new DeletedDeviceEnergyRecord(
            device.getType(),
            device.getRoomName(),
            device.getCurrentTotalEnergyConsumedKWh(),
            LocalDateTime.now()
        );
        currentUser.addDeletedDeviceRecord(energyRecord);

        // Remove device and update atomically
        currentUser.getGadgets().remove(device);
        return customerService.updateCustomer(currentUser);
    }
    return false;
}
```

PERFORMANCE AND SCALABILITY

Q38: How did you optimize database performance?

A:

Performance Optimizations:

1. Connection Pooling:

java

```
// DynamoDB Enhanced Client provides built-in connection pooling
enhancedClient = DynamoDbEnhancedClient.builder()
    .dynamoDbClient(dynamoDbClient)
    .build();
```

2. Batch Operations (could be implemented):

java

```
// Potential batch update implementation
public boolean updateMultipleCustomers(List<Customer> customers) {
    try {
        // DynamoDB supports batch write operations
        BatchWriteItemEnhancedRequest.Builder requestBuilder =
            BatchWriteItemEnhancedRequest.builder();

        for (Customer customer : customers) {
            requestBuilder.addPutItem(customerTable, customer);
        }

        enhancedClient.batchWriteItem(requestBuilder.build());
        return true;
    } catch (Exception e) {
        System.err.println("Batch update failed: " + e.getMessage());
        return false;
    }
}
```

3. Efficient Data Access:

- Single query retrieves all user data
- In-memory calculations for real-time values
- Minimal database round trips

4. Caching Strategy:

java

```
// Session-based caching
public class SessionManager {
    private Customer currentUser; // Cached user data
```

QUESTION AND ANSWERS

```
public void updateCurrentUser(Customer updatedUser) {
    this.currentUser = updatedUser; // Update cache
}

public Customer getCurrentUser() {
    return currentUser; // Return cached data
}
}
```

Scalability Considerations:

1. Partition Key Distribution: Email addresses provide natural distribution
2. Hot Partition Avoidance: User-based access patterns avoid concentrated writes
3. Read/Write Capacity: DynamoDB auto-scaling handles traffic spikes
4. Global Tables: Could enable multi-region deployment

Q39: How would you monitor and troubleshoot database issues in production?

A:

Monitoring Strategy:

1. Application-Level Monitoring:

```
java
public class DatabaseMetrics {
    private static final Logger logger = LoggerFactory.getLogger(DatabaseMetrics.class);

    public Customer findCustomerWithMetrics(String email) {
        long startTime = System.currentTimeMillis();
        try {
            Customer customer = findCustomerByEmail(email);
            long duration = System.currentTimeMillis() - startTime;

            logger.info("Customer query completed in {}ms for email: {}", duration, email);

            // Could send metrics to CloudWatch
            sendMetricToCloudWatch("DatabaseQueryDuration", duration);

            return customer;
        } catch (Exception e) {
            logger.error("Customer query failed for email: {} - Error: {}", email,
e.getMessage());
            sendMetricToCloudWatch("DatabaseQueryError", 1);
            throw e;
        }
    }
}
```

2. AWS CloudWatch Integration:

- Built-in DynamoDB Metrics: Read/write capacity, throttling, errors
- Custom Application Metrics: Query performance, business logic errors
- Alarms: Automated alerts for performance issues

3. Error Handling and Logging:

```
java
public boolean updateCustomerWithRetry(Customer customer) {
    int maxRetries = 3;
    int attempt = 0;

    while (attempt < maxRetries) {
        try {
            return updateCustomer(customer);
        } catch (ProvisionedThroughputExceededException e) {
            attempt++;
            logger.warn("DynamoDB throttling detected, attempt {} of {}", attempt, maxRetries);
        }
    }
}
```

QUESTION AND ANSWERS

```
        if (attempt < maxRetries) {
            // Exponential backoff
            int delay = (int) Math.pow(2, attempt) * 100;
            Thread.sleep(delay);
        }
    } catch (Exception e) {
        logger.error("Customer update failed permanently: {}", e.getMessage());
        return false;
    }
}

logger.error("Customer update failed after {} attempts", maxRetries);
return false;
}
```

Troubleshooting Tools:

1. Connection Testing: Built-in connection validation
2. Health Checks: Regular database connectivity tests
3. Performance Profiling: Query timing and resource utilization
4. Error Classification: Distinguish between network, auth, and data errors

CORE JAVA CONCEPTS

Q40: Explain the difference between JDK, JRE, and JVM.

A:

- JVM (Java Virtual Machine): Runtime environment that executes Java bytecode. Provides platform independence through bytecode interpretation/compilation.

- JRE (Java Runtime Environment): JVM + standard Java libraries + supporting files. Everything needed to run Java applications but not develop them.

- JDK (Java Development Kit): JRE + development tools (javac compiler, debugger, documentation tools). Everything needed to develop and run Java applications.

Relationship: JDK $\supset$ JRE $\supset$ JVM

In my project, I used Java 21 JDK for development, which includes all tools needed for compilation and the latest language features.

Q41: What are the key features of Java 21 that you used in your project?

A:

Java 21 LTS Features Used:

1. Enhanced Switch Expressions (from Java 14+):

java

```
public double getDefaultPowerRating(String deviceType) {
    return switch (deviceType.toUpperCase()) {
        case "TV" -> 150.0;
        case "AC" -> 1500.0;
        case "FAN" -> 75.0;
        case "MICROWAVE" -> 1200.0;
        default -> 50.0;
    };
}
```

2. Text Blocks (from Java 15+):

java

```
private void showPasswordRequirements() {
    String requirements = """
        Password Requirements:
        - 8-128 characters long
        - At least one uppercase letter (A-Z)
    """;
}
```

QUESTION AND ANSWERS

```
- At least one lowercase letter (a-z)
- At least one number (0-9)
- At least one special character (!@%$^&)
""";
System.out.println(requirements);
}
```

3. Pattern Matching for instanceof (from Java 16+):

```
java
public void processAutomationAction(Object action) {
    if (action instanceof SceneAction sceneAction) {
        executeSceneAction(sceneAction);
    } else if (action instanceof TimerTask timerTask) {
        executeTimerTask(timerTask);
    }
}
```

4. Records (from Java 17+):

```
java
public record DeviceSuggestion(String deviceType, String action, String reason, Priority
priority) {
    public boolean isHighPriority() {
        return priority == Priority.HIGH;
    }
}
```

Q42: Explain Object-Oriented Programming principles with examples from your project.

A:

1. Encapsulation:

```
java
public class Gadget {
    private String type;           // Hidden implementation
    private double powerRatingWatts; // Private data
    private LocalDateTime lastOnTime; // Internal state

    // Controlled access through public methods
    public void turnOn() {
        if (!isOn()) {
            this.lastOnTime = LocalDateTime.now();
            updateUsageAndEnergy(); // Internal method call
        }
        this.status = GadgetStatus.ON.name();
    }

    // Private helper method - implementation detail hidden
    private void updateUsageAndEnergy() {
        // Complex energy calculation logic hidden from clients
    }
}
```

2. Inheritance:

```
java
// Base service class
public abstract class BaseService {
    protected final Logger logger = LoggerFactory.getLogger(getClass());

    protected void logOperation(String operation, boolean success) {
        if (success) {
            logger.info("Operation succeeded: {}", operation);
        } else {
            logger.error("Operation failed: {}", operation);
        }
    }
}

// Specific service inheriting common functionality
```


QUESTION AND ANSWERS

```
public class CustomerService extends BaseService {
    public boolean registerCustomer(String name, String email, String password) {
        boolean success = performRegistration(name, email, password);
        logOperation("Customer Registration", success); // Inherited method
        return success;
    }
}
```

3. Polymorphism:

java

```
// Interface defining contract
public interface Executable {
    boolean execute();
    String getDescription();
}

// Different implementations
public class SceneExecution implements Executable {
    public boolean execute() {
        return executeScene();
    }
    public String getDescription() { return "Scene execution"; }
}

public class TimerExecution implements Executable {
    public boolean execute() {
        return executeTimer();
    }
    public String getDescription() { return "Timer execution"; }
}

// Polymorphic usage
public void executeTask(Executable task) {
    System.out.println("Executing: " + task.getDescription());
    boolean result = task.execute(); // Polymorphic method call
    System.out.println("Result: " + (result ? "Success" : "Failed"));
}
```

4. Abstraction:

java

```
// Abstract class defining template
public abstract class DeviceController {
    // Template method - defines algorithm structure
    public final boolean controlDevice(String action) {
        if (!validateAction(action)) return false;
        if (!checkPermissions()) return false;

        boolean result = executeDeviceAction(action); // Abstract method
        logDeviceAction(action, result);
        return result;
    }

    // Abstract method - subclasses provide implementation
    protected abstract boolean executeDeviceAction(String action);

    // Concrete methods - common functionality
    private boolean validateAction(String action) {
        return "ON".equals(action) || "OFF".equals(action);
    }
}
```

COLLECTIONS AND GENERICS

Q43: Compare different Java collections and explain when to use each.

A:

Collections Used in My Project:

1. ArrayList - Dynamic device lists:

```
java
public class Customer {
    private List<Gadget> gadgets = new ArrayList<>(); // Ordered, allows duplicates, indexed access

    public void addGadget(Gadget gadget) {
        // ArrayList provides O(1) append, good for sequential access
        this.gadgets.add(gadget);
    }
}
```

2. HashMap - Fast lookups:

```
java
public class SmartScenesService {
    // HashMap provides O(1) average lookup time
    private final Map<String, List<SceneAction>> predefinedScenes = new HashMap<>();
    private final Map<String, Map<String, List<SceneAction>>> userCustomScenes = new HashMap<>();

    public List<SceneAction> getSceneActions(String sceneName) {
        return predefinedScenes.get(sceneName.toUpperCase()); // O(1) lookup
    }
}
```

3. Arrays.asList() - Fixed-size lists:

```
java
private static final List<String> COMMON_PASSWORDS = Arrays.asList(
    "password", "123456", "password123", "admin", "qwerty"
); // Immutable list from array
```

Collection Comparison:

Collection	Use Case	Time Complexity	My Project Usage
ArrayList	Ordered, indexed access	O(1) access, O(n) search	Device lists, group members
LinkedList	Frequent insertions/deletions	O(1) insert/delete, O(n) access	Not used (ArrayList sufficient)
HashMap	Key-value mapping	O(1) average lookup	Scene mappings, user caches
TreeMap	Sorted key-value pairs	O(log n) operations	Could use for sorted device lists
HashSet	Unique elements, fast lookup	O(1) average operations	Could use for unique device types

Q44: Explain generics and type safety with examples.

A:

Generic Collections:

```
java
// Type-safe collections with generics
public class AlertService {
    private final Map<String, List<Alert>> userAlerts = new HashMap<>();
    //      ↑      ↑      ↑
    //      Key Type Value Type Element Type

    public List<Alert> getUserAlerts(String userEmail) {
        return userAlerts.computeIfAbsent(userEmail, k -> new ArrayList<>());
        //      ↑ Compile-time type checking ensures List<Alert> return type
    }
}
```

Generic Methods:

```
java
public class ServiceUtils {
    // Generic method with type parameter T
    public static <T> List<T> filterNonNull(List<T> input) {
        return input.stream()
            .filter(Objects::nonNull) // Type-safe filtering
            .collect(Collectors.toList());
    }

    // Usage with different types
    List<Gadget> validGadgets = filterNonNull(allGadgets);
    List<Customer> validCustomers = filterNonNull(allCustomers);
}
```

Bounded Generics:

```
java
// Upper bounded wildcard
public class EnergyCalculator {
    // Method accepts List of Gadget or any subtype
    public double calculateTotalEnergy(List<? extends Gadget> devices) {
        return devices.stream()
            .mapToDouble(Gadget::getCurrentTotalEnergyConsumedKWh)
            .sum();
    }
    // Lower bounded wildcard
    public void addDevicesToList(List<? super Gadget> deviceList, Gadget device) {
        deviceList.add(device); // Can add Gadget to list of Gadget supertypes
    }
}
```

Benefits of Generics:

- Compile-time Type Safety: Catches type errors at compilation
- Elimination of Casts: No need for explicit casting
- Code Reusability: Same code works with different types
- Performance: No runtime type checking overhead

EXCEPTION HANDLING AND BEST PRACTICES

Q45: How do you handle exceptions in your application?

A:

Exception Handling Strategy:

1. Specific Exception Handling:

```
java
public boolean scheduleDeviceTimer(String deviceType, String roomName, String action, String
dateTimeStr) {
    try {
        LocalDateTime scheduledTime = parseDateTime(dateTimeStr);

        if (scheduledTime.isBefore(LocalDateTime.now())) {
            System.out.println("Cannot schedule timer in the past!");
            return false;
        }

        // Business logic here
        return updateDeviceSchedule(deviceType, roomName, action, scheduledTime);
    } catch (DateTimeParseException e) {
        System.out.println("[ERROR] Invalid date format: " + e.getMessage());
        return false;
    } catch (IllegalArgumentException e) {
        System.out.println("[ERROR] Invalid parameters: " + e.getMessage());
        return false;
    } catch (Exception e) {
        System.err.println("[ERROR] Unexpected error scheduling timer: " + e.getMessage());
        return false;
    }
}
```

```
    }
}
```

2. Resource Management with Try-with-Resources:

java

```
private static void loadProperties() {
    properties = new Properties();
    try (InputStream input = DynamoDBConfig.class.getClassLoader()
        .getResourceAsStream("application.properties")) {

        if (input != null) {
            properties.load(input);
        }
    } catch (IOException e) {
        System.err.println("Error loading properties: " + e.getMessage());
        // Set default properties
        properties.setProperty("dynamodb.local", "true");
    }
}
```

3. Custom Exception Handling:

java

```
public class DeviceNotFoundException extends Exception {
    private final String deviceType;
    private final String roomName;

    public DeviceNotFoundException(String deviceType, String roomName) {
        super(String.format("Device not found: %s in %s", deviceType, roomName));
        this.deviceType = deviceType;
        this.roomName = roomName;
    }

    public String getDeviceType() { return deviceType; }
    public String getRoomName() { return roomName; }
}

// Usage
public void controlDevice(String type, String room, String action) throws
DeviceNotFoundException {
    Gadget device = findGadget(type, room);
    if (device == null) {
        throw new DeviceNotFoundException(type, room);
    }
    // Control device logic
}
```

4. Graceful Degradation:

java

```
public CustomerService() {
    DynamoDbEnhancedClient enhancedClient = DynamoDBConfig.getEnhancedClient();
    if (enhancedClient != null) {
        this.customerTable = enhancedClient.table("customers",
TableSchema.fromBean(Customer.class));
        this.isDemoMode = false;
    } else {
        // Graceful fallback to in-memory storage
        System.out.println("[INFO] Running in DEMO MODE - data won't persist");
        this.customerTable = null;
        this.isDemoMode = true;
        this.demoCustomers = new HashMap<>();
    }
}
```

Exception Handling Best Practices:

- Specific Exceptions: Catch specific exceptions rather than generic Exception

QUESTION AND ANSWERS

- Resource Cleanup: Use try-with-resources for automatic resource management
- Graceful Degradation: Provide fallback mechanisms when possible
- User-Friendly Messages: Convert technical exceptions to user-readable messages
- Logging: Log exceptions for debugging while showing clean messages to users

Q46: Explain the difference between checked and unchecked exceptions.

A:

Checked Exceptions (Compile-time enforced):

java

```
// Must be declared in method signature or handled
public void saveCustomerData(Customer customer) throws IOException, SQLException {
    // Checked exceptions must be handled
    try (FileWriter writer = new FileWriter("customer.dat")) {
        writer.write(customer.toString());
    } // IOException automatically handled by try-with-resources
}

// Caller must handle or declare
public void processCustomer(Customer customer) {
    try {
        saveCustomerData(customer);
    } catch (IOException | SQLException e) {
        System.err.println("Failed to save customer: " + e.getMessage());
    }
}
```

Unchecked Exceptions (Runtime exceptions):

java

```
public Gadget findGadget(String type, String roomName) {
    if (type == null || roomName == null) {
        // Unchecked exception - not required to be declared
        throw new IllegalArgumentException("Device type and room name cannot be null");
    }

    return gadgets.stream()
        .filter(g -> g.getType().equals(type) && g.getRoomName().equals(roomName))
        .findFirst()
        .orElse(null); // Could throw NoSuchElementException (unchecked)
}
```

When to Use Each:

- Checked Exceptions: For recoverable conditions (network failures, file I/O)
- Unchecked Exceptions: For programming errors (null pointers, illegal arguments)

.

MULTITHREADING AND CONCURRENCY

Q47: How did you handle concurrency in your application?

A:

Thread Pool Usage:

java

```
public class TimerService {
    private final ScheduledExecutorService scheduler;

    private TimerService(CustomerService customerService) {
        // Fixed thread pool for timer execution
        this.scheduler = Executors.newScheduledThreadPool(5);
        startTimerMonitoring();
    }

    private void startTimerMonitoring() {
        // Background task running every 30 seconds
        scheduler.scheduleAtFixedRate(
            this::checkAndExecuteDueTimers,
            0, // Initial delay
            30, // Period
            TimeUnit.SECONDS);
    }
}
```

```

        TimeUnit.SECONDS
    );
}

public void shutdown() {
    scheduler.shutdown();
    try {
        if (!scheduler.awaitTermination(60, TimeUnit.SECONDS)) {
            scheduler.shutdownNow();
        }
    } catch (InterruptedException e) {
        scheduler.shutdownNow();
        Thread.currentThread().interrupt();
    }
}
}
}

```

Thread Safety with Singleton Pattern:**java**

```

public class SessionManager {
    private static volatile SessionManager instance;
    private volatile Customer currentUser;

    // Double-checked locking for thread-safe singleton
    public static SessionManager getInstance() {
        if (instance == null) {
            synchronized (SessionManager.class) {
                if (instance == null) {
                    instance = new SessionManager();
                }
            }
        }
        return instance;
    }

    public synchronized void login(Customer customer) {
        this.currentUser = customer;
    }

    public synchronized void logout() {
        this.currentUser = null;
    }

    public synchronized boolean isLoggedIn() {
        return currentUser != null;
    }
}

```

Concurrent Collections:**java**

```

public class AlertService {
    // Thread-safe map for concurrent access
    private final ConcurrentHashMap<String, List<Alert>> userAlerts = new ConcurrentHashMap<>();

    public void addAlert(String userEmail, Alert alert) {
        // Atomic operation - thread-safe
        userAlerts.computeIfAbsent(userEmail, k -> new CopyOnWriteArrayList<>()).add(alert);
    }

    public List<Alert> getUserAlerts(String userEmail) {
        return userAlerts.getDefault(userEmail, Collections.emptyList());
    }
}

```

Q48: Explain different ways to create threads in Java.

A:

1. Extending Thread Class:

java

```
public class DeviceMonitorThread extends Thread {
    private final List<Gadget> devices;
    private volatile boolean running = true;

    public DeviceMonitorThread(List<Gadget> devices) {
        this.devices = devices;
        setName("DeviceMonitor");
    }

    @Override
    public void run() {
        while (running) {
            for (Gadget device : devices) {
                checkDeviceHealth(device);
            }
            try {
                Thread.sleep(60000); // Check every minute
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                break;
            }
        }
    }

    public void stopMonitoring() {
        running = false;
        interrupt();
    }
}
```

2. Implementing Runnable Interface:

java

```
public class EnergyCalculationTask implements Runnable {
    private final Customer customer;
    private final CountDownLatch latch;

    public EnergyCalculationTask(Customer customer, CountDownLatch latch) {
        this.customer = customer;
        this.latch = latch;
    }

    @Override
    public void run() {
        try {
            calculateCustomerEnergyUsage(customer);
        } finally {
            latch.countDown(); // Signal completion
        }
    }
}

// Usage
CountDownLatch latch = new CountDownLatch(customers.size());
ExecutorService executor = Executors.newFixedThreadPool(10);

for (Customer customer : customers) {
    executor.submit(new EnergyCalculationTask(customer, latch));
}

latch.await(); // Wait for all tasks to complete
executor.shutdown();
```

3. Using Callable and Future:

```
java
public class WeatherDataFetcher implements Callable<WeatherData> {
    private final String location;

    public WeatherDataFetcher(String location) {
        this.location = location;
    }

    @Override
    public WeatherData call() throws Exception {
        // Simulate API call
        Thread.sleep(2000);
        return new WeatherData("Sunny", 25.0, 60, 10.0, "Clear sky", 50);
    }
}

// Usage
ExecutorService executor = Executors.newSingleThreadExecutor();
Future<WeatherData> future = executor.submit(new WeatherDataFetcher("Delhi"));

try {
    WeatherData weather = future.get(5, TimeUnit.SECONDS); // Timeout after 5 seconds
    processWeatherData(weather);
} catch (TimeoutException e) {
    System.out.println("Weather data fetch timed out");
    future.cancel(true);
}
```

4. Lambda Expressions with Executors:

```
java
// Modern approach using lambda expressions
public void processAlertsAsync(List<Alert> alerts) {
    ExecutorService executor = Executors.newCachedThreadPool();

    for (Alert alert : alerts) {
        executor.submit(() -> {
            try {
                processAlert(alert);
            } catch (Exception e) {
                System.err.println("Error processing alert: " + e.getMessage());
            }
        });
    }

    executor.shutdown();
}
```

JAVA 8+ FEATURES AND FUNCTIONAL PROGRAMMING**Q49: How did you use Java 8+ features in your project?**

A:

1. Stream API:

```
java
// Filter and transform device collections
public List<Gadget> getHighEnergyDevices(Customer customer) {
    return customer.getGadgets().stream()
        .filter(device -> device.getCurrentTotalEnergyConsumedKWh() > 10.0)
        .filter(device -> device.isOn())

    .sorted(Comparator.comparingDouble(Gadget::getCurrentTotalEnergyConsumedKWh).reversed())
    .collect(Collectors.toList());
}

// Group devices by room
public Map<String, List<Gadget>> groupDevicesByRoom(List<Gadget> devices) {
    return devices.stream()
        .collect(Collectors.groupingBy(Gadget::getRoomName));
}
```


QUESTION AND ANSWERS

```
}

// Calculate total energy consumption
public double calculateTotalEnergyConsumption(List<Gadget> devices) {
    return devices.stream()
        .mapToDouble(Gadget::getCurrentTotalEnergyConsumedKWh)
        .sum();
}
```

2. Optional Class:

```
java
public Optional<Gadget> findDeviceOptionally(String type, String roomName) {
    return gadgets.stream()
        .filter(g -> g.getType().equalsIgnoreCase(type))
        .filter(g -> g.getRoomName().equalsIgnoreCase(roomName))
        .findFirst();
}

// Usage with Optional
public boolean controlDevice(String type, String roomName, String action) {
    return findDeviceOptionally(type, roomName)
        .map(device -> {
            if ("ON".equals(action)) device.turnOn();
            else device.turnOff();
            return true;
        })
        .orElse(false);
}
```

3. Method References:

```
java
// Method reference examples
public List<String> getDeviceTypes(List<Gadget> devices) {
    return devices.stream()
        .map(Gadget::getType) // Method reference
        .distinct()
        .sorted(String::compareToIgnoreCase) // Static method reference
        .collect(Collectors.toList());
}

// Constructor reference
public List<Customer> createCustomersFromData(List<String[]> customerData) {
    return customerData.stream()
        .map(data -> new Customer(data[0], data[1], data[2])) // Constructor reference
        .collect(Collectors.toList());
}
```

4. Functional Interfaces:

```
java
// Custom functional interface
@FunctionalInterface
public interface DeviceValidator {
    boolean validate(Gadget device);
}

// Usage with lambda expressions
public List<Gadget> filterDevices(List<Gadget> devices, DeviceValidator validator) {
    return devices.stream()
        .filter(validator::validate)
        .collect(Collectors.toList());
}

// Different validation strategies
DeviceValidator highEnergyValidator = device -> device.getPowerRatingWatts() > 500;
DeviceValidator recentlyUsedValidator = device -> device.getLastOnTime() != null &&
    device.getLastOnTime().isAfter(LocalDate.now().minusHours(24));

List<Gadget> highEnergyDevices = filterDevices(allDevices, highEnergyValidator);
List<Gadget> recentDevices = filterDevices(allDevices, recentlyUsedValidator);
```

Q50: What are the advantages of using functional programming concepts?

A:

Advantages Demonstrated in My Project:

1. Immutability and Side-Effect Free Functions:

java

```
// Pure function - no side effects
public static double calculateEnergyCost(double energyKWh) {
    return energyKWh <= 30 ? energyKWh * 1.90 :
        energyKWh <= 75 ? 30 * 1.90 + (energyKWh - 30) * 3.0 :
        energyKWh <= 125 ? 30 * 1.90 + 45 * 3.0 + (energyKWh - 75) * 4.50 :
        / ... more slab calculations /;
}
```

2. Higher-Order Functions:

java

```
// Function that takes another function as parameter
public void processDevices(List<Gadget> devices, Consumer<Gadget> processor) {
    devices.forEach(processor);
}

// Usage with different processors
processDevices(devices, device -> device.turnOff()); // Lambda
processDevices(devices, this::updateDeviceHealth);    // Method reference
processDevices(devices, device -> logDeviceStatus(device)); // Custom logic
```

3. Declarative vs Imperative Style:

java

```
// Imperative approach (traditional)
public List<String> getActiveDeviceNamesImperative(List<Gadget> devices) {
    List<String> activeDevices = new ArrayList<>();
    for (Gadget device : devices) {
        if (device.isOn()) {
            String deviceName = device.getType() + " in " + device.getRoomName();
            activeDevices.add(deviceName);
        }
    }
    Collections.sort(activeDevices);
    return activeDevices;
}

// Functional approach (declarative)
public List<String> getActiveDeviceNamesFunctional(List<Gadget> devices) {
    return devices.stream()
        .filter(Gadget::isOn)
        .map(device -> device.getType() + " in " + device.getRoomName())
        .sorted()
        .collect(Collectors.toList());
}
```

Benefits:

- Readability: Code expresses what you want, not how to do it
- Maintainability: Less complex state management
- Testability: Pure functions are easier to test
- Parallelization: Functional operations can be easily parallelized
- Reusability: Higher-order functions promote code reuse

This completes the comprehensive interview Q&A covering all aspects of the IoT Smart Home Dashboard project and general Java concepts.

Q: "Why did you choose DynamoDB over a relational database?"

A: "IoT systems have simple access patterns - all data is accessed by user. DynamoDB's key-value structure perfectly matches this pattern, provides automatic scalability, and eliminates the overhead of complex joins. For an IoT system that might scale to millions of devices, DynamoDB's performance characteristics are superior."

Q: "How would you handle system failure scenarios?"

A: "I implemented multiple layers of error handling - input validation, business rule validation, and exception handling. The system gracefully degrades, provides clear error messages, and maintains data consistency. For production, I'd add circuit breakers, retry logic, and comprehensive logging."

Q: "What design patterns did you use and why?"

A: "Singleton for system services to ensure single point of control, Factory for device creation to support extensibility, Observer for real-time alerts to enable event-driven architecture, and Strategy for energy optimization to support different algorithms."

Q: "How would you scale this system?"

A: "The service-oriented architecture makes horizontal scaling straightforward. Each service can be deployed independently, DynamoDB auto-scales, and the stateless design supports load balancing. I'd implement API gateways, microservices deployment, and distributed caching for enterprise scale."

Q: "What security considerations did you implement?"

A: "Multi-layered security: BCrypt password hashing, input validation, session management, SQL injection prevention, and memory clearing for sensitive data. For production, I'd add OAuth2, rate limiting, and comprehensive audit logging."